

and the multinomial distribution for categorical features with k classes (K. P. Murphy, 2012, section 3.5).

Once the model parameters have been estimated from the training data, the naive Bayes classifier assigns a feature vector $\mathbf{x}$ to class C_l when the numerator of (12.1) is maximal for class C_l (the denominator, being a normalization factor, is ignored), that is, the model outputs $\hat{y} = C_l$, where

$$P(C_l) \prod_{j=1}^D p(x_j|C_l) \geq P(C_i) \prod_{j=1}^D p(x_j|C_i), \quad \text{for all } i \neq l. \quad (12.45)$$

As an example, let us apply naive Bayes to the same forest classification dataset in Fig. 12.1(a). Classification of the training data by naive Bayes assuming Gaussian distribution is shown in Fig. 12.1(d). The assumption that the features are conditionally independent restricts the Gaussian ellipses to have their semi-major and semi-minor axes oriented parallel to the x_1 and x_2 axes. The misclassification rates are 13.1% for the training data and 20.9% for the test data, quite comparable to the LDA values of 14.1% (training) and 19.7% (test) from the previous section.

12.4 K -nearest Neighbours

In the previous two sections, we have presented two simple parametric classifiers, namely LDA and naive Bayes, which fit parametric models to the training data. *Parametric models* are defined by a finite number of parameters, whereas the number of parameters in *non-parametric models* grow with the amount of training data. Parametric models tend to be simpler and faster than non-parametric models, but are constrained by the assumed functional form.

In this section, we will study a simple non-parametric model for classification and regression called the *K-nearest neighbours* (KNN),³ first proposed by Cover and Hart (1967). As a *memory-based* classifier, KNN stores all the training data in memory and does not fit any model to the training data. Being non-parametric, KNN is better than parametric models when the decision boundaries are complicated, but can be very expensive with large datasets.

The basic idea of KNN is extremely simple: given a D -dimensional feature vector $\mathbf{x}$, one finds K feature vectors closest to $\mathbf{x}$ in the training dataset based on some distance measure (e.g. the Euclidean distance). For classification, the class $\hat{y}$ to assign to $\mathbf{x}$ is decided by simple voting from the classes in the K nearest neighbours. For example, assume $\mathbf{x}$ is a feature vector containing various meteorological measurements and y is the weather class of the next day, with three classes, ‘dry’, ‘rainy’ and ‘snowy’. Suppose KNN with $K = 7$ is used to classify the dataset in Fig. 12.3. Among the seven nearest neighbours around the point $\mathbf{x}$, four are labelled ‘dry’, one ‘rainy’ and two ‘snowy’; thus, ‘dry’ receives 4 votes, ‘rainy’ 1 and ‘snowy’ 2 votes, so the model output is ‘dry’. If

³ Be careful not to confuse the classification method K -nearest neighbours with K -means clustering, a clustering method under unsupervised learning (Section 10.2.1).

the nearest neighbours happen to be 3 ‘dry’, 1 ‘rainy’ and 3 ‘snowy’, we have a tie in the voting. How do we break the tie? One option is to just choose randomly among the tied top classes. Another option is to use the closest neighbour as the tie-breaker. To avoid ties, KNN is often run with K being an odd integer.

For regression, KNN outputs the mean of the y values of the K nearest neighbours.

More formally, for the KNN model, we can define a posterior probability

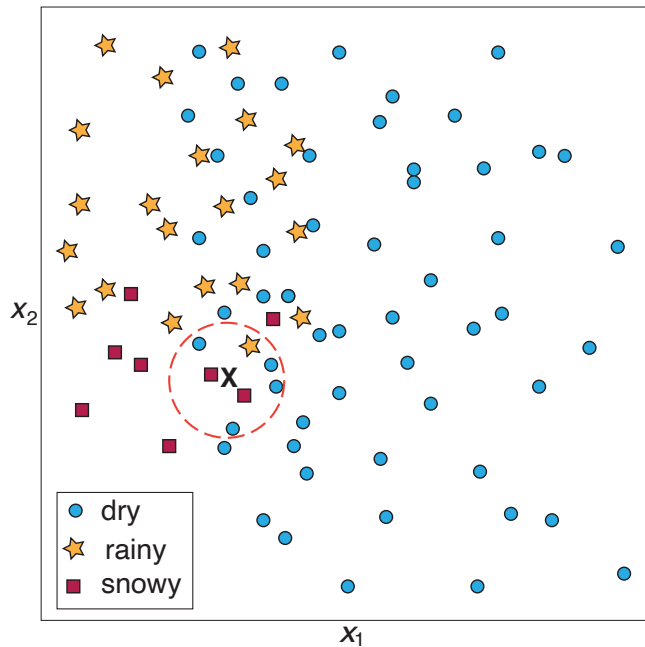
$$P(C_i|\mathbf{x}) = \frac{1}{K} \sum_{j \in N_K(\mathbf{x})} \delta(y_j, C_i), \quad (12.46)$$

where the K nearest neighbours in the set $N_K(\mathbf{x})$ have class labels $\{y_j\}$ and

$$\delta(u, v) = \begin{cases} 1 & \text{if } u = v \\ 0 & \text{if } u \neq v. \end{cases} \quad (12.47)$$

In other words, the summation in (12.46) simply counts the number of data points among the K nearest neighbours that belong to class C_i . KNN outputs $\hat{y} = C_l$ where $P(C_l|\mathbf{x}) \geq P(C_i|\mathbf{x})$ for all $i \neq l$.

Figure 12.3 Illustrating seven nearest neighbours (within the dashed circle) around a particular feature vector $\mathbf{x}$ in a 2-D feature space for the KNN classifier with $K = 7$. The three classes represent ‘dry’, ‘rainy’ and ‘snowy’ conditions. Since ‘dry’ has the most votes in this neighbourhood, the model outputs ‘dry’.



There are many choices for the distance measure. When using the Euclidean distance, it is commonly recommended that one standardizes each feature variable, especially if different features are measured in different units.

For KNN to work well, one needs to choose an optimal value for the hyper-parameter K . A common ‘rule of thumb’ is to choose $K = \sqrt{N}$, with N being

the sample size of the training dataset. Lall and Sharma (1996) supported this choice for $1 \leq D \leq 6$ and $N \geq 100$. To avoid having to choose one value for K , Hassanat et al. (2014) proposed using an ensemble of KNN models, with $K = 1, 3, 5, \dots, K_{\max}$, where $K_{\max}$ is the largest odd integer $\leq \sqrt{N}$. Each ensemble member votes for a particular class, and the ensemble model gathers all the votes from its members and chooses the class with the most votes. Cross-validation can also be used to find the optimal value for K .

When the classes are imbalanced, for example, there are far more ‘dry’ days than ‘snowy’ days, KNN tends to ignore the infrequent class. For instance in Fig. 12.3, the two closest neighbours to $\mathbf{x}$ are ‘snowy’, but the KNN outputs ‘dry’ since there are four ‘dry’ points among the seven nearest neighbours. One way to deal with imbalanced classes is to use weighted voting, that is, when summing up the votes, multiply each vote by a weight factor proportional to the inverse of the distance between $\mathbf{x}$ and the nearest neighbour.

Figure 12.4 shows the results from KNN for the forest classification dataset using three different values for K . The decision boundaries in KNN are complex when compared with those in LDA, logistic regression and naive Bayes (Fig. 12.1). In general, using a larger K tends to produce smoother decision boundaries. The misclassification rates for the training data are 4.0% ($K = 1$), 13.6% ($K = 11$), 13.6% ($K = 31$) and, for the test data, 25.9% ($K = 1$), 18.2% ($K = 11$), 21.5% ($K = 31$). Using $K = 1$ overfitted the training data, leading to poorer performance on the test data, while $K = 31$ also underperformed $K = 11$ on the test data.

Let us summarize the pros and cons for KNN:

KNN is very easy to understand and simple to implement. As there is no model to be trained, when new data become available there is no need to retrain the model. Being non-parametric, KNN makes no assumption about the underlying distribution of the data.

On the negative side, KNN needs to store all data in memory and can be expensive when making predictions. KNN suffers from the ‘curse of dimensionality’ (Section 1.5) when D , the dimension of the feature vector, grows. One might try to reduce D by some dimension reduction technique such as principal component analysis.

12.5 Extreme Learning Machine Classifier

The extreme learning machine (ELM), a non-linear neural network model with random weights in its hidden layer, which requires only the solution of a linear least squares problem, has been introduced in Section 6.4 for regression problems. Here, we will use ELM for classification. The model is run identical to the regression problem except that the target data are coded using *one-hot encoding*. For example, consider the forest cover dataset containing four classes of land cover, $C_1, \dots, C_4$. If an input $\mathbf{x}$ belongs to class C_1 , the target is coded by the binary vector $[1 \ 0 \ 0 \ 0]$, whereas $[0 \ 1 \ 0 \ 0]$ is used for C_2 , $[0 \ 0 \ 1 \ 0]$ for C_3 and $[0 \ 0 \ 0 \ 1]$ for C_4 . In general, with k classes, instead of the target being an

Deep Learning

The traditional multi-layer perceptron (MLP) neural network (NN) model seen previously in Section 6.3 is commonly run with only one layer of hidden neurons, because using two hidden layers often fails to significantly improve the model accuracy on test data. Adding even more hidden layers usually leads to a drop in model accuracy, so in practice the traditional MLP NN is mostly limited to one or two hidden layers. This was very frustrating to the NN researchers, since biological neural networks are certainly not limited to so few layers. The problem lies in using gradient-based back-propagation for optimization, where the gradients (error signals) can become vanishingly small after propagating through many layers of neurons. This is known as the *vanishing gradient problem*, first explained by Sepp Hochreiter in his diplom thesis of 1991 (written in German). Without a solution in sight, NN research stalled while newer methods – kernel methods (e.g. support vector machines) emerging from the mid-1990s and random forests from 2001 – seriously challenged NN’s dominant position in ML.

NN models with more hidden layers than the traditional NN are referred to as *deep neural network* (DNN) or *deep learning* (DL) models. Reviews of DL are given by LeCun, Bengio, et al. (2015), Schmidhuber (2015), and Ponti et al. (2017). The Venn diagram in Fig. 15.1 illustrates the relation between DL, NN, ML, AI, and so on, with $DL \subset NN \subset ML \subset AI$.

Interest in deep feedforward networks was revived around 2006 by a group of researchers brought together by the Canadian Institute for Advanced Research (CIFAR) (LeCun, Bengio, et al., 2015). As DNN models were difficult to train with back-propagation, the early DL models first pre-trained the network using unsupervised learning so the network weights were already in a reasonable range before using back-propagation (Hinton, Osindero, et al., 2006). These early deep networks, called *deep belief networks*, have since fallen out of favour but were the first NN models demonstrating that deep networks can be trained successfully (Goodfellow, Bengio, et al., 2016, section 20.3). The deep belief networks have been replaced by newer DNN models that do not require pre-training by unsupervised learning.

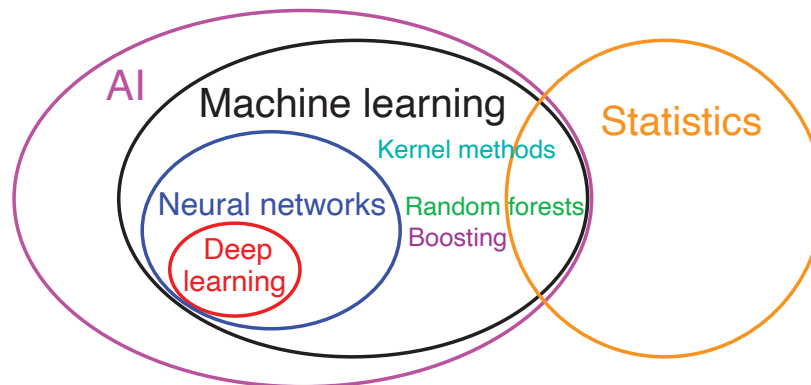


Figure 15.1 A schematic Venn diagram illustrating the relation between AI, statistics, machine learning, neural networks and deep learning, as well as kernel methods (Chapter 13), random forests and boosting (Chapter 14). [Reproduced from Hsieh (2022).]

The ImageNet project is a large database of images for use in computer vision research. During 2010–2017, the ImageNet project ran a series of annual ML contests, the ImageNet Large Scale Visual Recognition Challenge (ILSVRC), where various ML models competed in classifying and detecting objects and scenes from images (Russakovsky et al., 2015).

The first time DNN made a monumental impact was in the ILSVRC 2012 competition, where the AlexNet DNN model with eight layers (Krizhevsky et al., 2012), submitted by the Supervision group from the University of Toronto, won the competition, surpassing previous year’s records by wide margins. This breakthrough started the exponential growth in DNN research worldwide. By ILSVRC 2014, the winning DNN model, GooLeNet from Google, had 22 layers. By ILSVRC 2015, the winning DNN model, from the Microsoft Research Lab in Beijing, had 152 layers (He et al., 2016). In theory, an MLP NN with a single hidden layer is enough to represent any continuous function (Section 6.3.2); in practice, a model with more hidden layers (and modern training approach) tends to outperform one with fewer layers, as explained by Montufar et al. (2014) and Goodfellow, Bengio, et al. (2016, section 6.4.1).

A common question on terminology: How many layers of neurons does an NN model need in order to be considered a *deep* NN? Most leading researchers consider DNN models as having $\gtrsim 5$ layers of mapping functions with adjustable weights (i.e. $\gtrsim 4$ or more hidden layers) (LeCun, Bengio, et al., 2015, p. 438; Montufar et al., 2014). Unfortunately, the term ‘deep NN’ or ‘deep learning’ has been watered down by some researchers to describe any NN model with more than one hidden layer. The author has been frustrated from attending conference presentations where the ‘deep learning’ advertised in the abstract turned out to be a traditional two-hidden layer MLP NN!

A number of technical advances made DNN possible:

- [1] The vanishing gradient problem found in NN models using sigmoidal-shaped activation functions (e.g. the hyperbolic tangent function) was overcome by new types of activation functions, for example the *rectified linear unit* (ReLU). The ReLU function f , defined by

$$f(x) = \max(0, x) \quad (15.1)$$

(Fig. 6.3(d)), has been found to be much better than the sigmoidal-shaped activation functions in the training of DNN models (Nair and Hinton, 2010; Glorot et al., 2011). The hyperbolic tangent function has gradients in the range $(-1, 1)$, and as $x \rightarrow \pm\infty$, the gradient of $\tanh x \rightarrow 0$. As back-propagation computes gradients using the chain rule, in a network with many layers, this could lead to multiplying many numbers of small magnitude to obtain the gradients of the early layers. In contrast, the ReLU function has gradient = 1 for $x > 0$, allowing it to circumvent the vanishing gradient problem in DNN.¹

There are now several activation functions which have evolved from the original ReLU function, for example the leaky ReLU, the parametric ReLU, the softplus (a smooth version of ReLU), the swish function, and so on, with a comparison of their performance given by Ramachandran et al. (2017). The best performer seems to be the *swish function* (Fig. 6.3(d)), defined by

$$f(x) = x \cdot \text{sigmoid}(\beta x) = \frac{x}{1 + e^{-\beta x}}, \quad (15.2)$$

where β can be simply set to the constant 1 or be treated as a tuneable hyperparameter.

- [2] The traditional MLP NN has each neuron in one layer connected to all the neurons in the preceding layer. When working with image data, the MLP uses a huge number of weights – for example mapping an input 100×100 image to just one neuron in the first hidden layer requires 10,000 weights! Since in nature, neurons are connected only to neighbouring neurons, to have every neuron in one layer of an NN model connected to all the neurons in the preceding layer appears unnatural and very wasteful of computing resources. Inspired by biological studies in visual cortexes, convolutional layers were first introduced in the ‘neocognitron’ model (Fukushima, 1975; Fukushima, 1980). In *convolutional layers*, a neuron is only connected to a small patch of neurons in the preceding layer, thereby resulting in a drastic reduction of weights compared to the traditional fully-connected

¹ A caveat is that ReLU is designed for deep NN – in a shallow NN, the non-smooth ReLU activation function is likely to underperform the smooth sigmoidal-shaped activation functions. In NN research, ReLU dates back to at least Fukushima (1975) but has not been able to compete against sigmoidal-shaped activation functions in shallow NN models (Goodfellow, Bengio, et al., 2016, p. 219).

layers, making deep NN models practical.² The modern *convolutional neural network* (CNN), first developed by LeCun, B. Boser, et al. (1989), is widely used in DL.

- [3] The regularization technique *dropout* was introduced by Hinton, Srivastava, et al. (2012) to alleviate overfitting in DNN (Section 8.8).
- [4] *Batch normalization* (Ioffe and Szegedy, 2015), which normalizes (i.e. standardizes) the inputs at each layer during training, was found to stabilize and accelerate DNN training.

With all the impressive breakthroughs in DL, the reader may start to believe DNN models to be invincible against all other ML models. That belief is not correct, since the best ML model is very problem dependent. For instance, a farmer would find a rifle perfect for defending against wolves or bears, but useless against locusts or caterpillars. If a dataset has only a small number of data points, it is possible for even linear regression to be the best model.

There are two main types of datasets, structured and unstructured. *Structured* datasets have a tabular format, for example like an Excel spreadsheet, with the variables listed in columns. In contrast, *unstructured* datasets include images, videos, audio, text, and so on. For unstructured datasets, DL has indeed been dominant. For structured datasets, however, ‘shallow’ ML methods, mainly gradient boosting methods such as XGBoost, have often beaten DL methods in competitions, for example those organized by Kaggle (www.kaggle.com).

How can ‘shallow’ boosting beat deep NN in structured data? Typically, in a structured dataset, the predictors are quite inhomogeneous (e.g. pressure, temperature, humidity, etc.), whereas in an unstructured dataset, the predictors are more homogeneous (e.g. temperature at various pixels in an image or at various grid points in a numerical model). Boosting is based on decision trees (Chapter 14), where the effects of the predictors are treated independently of each other, as the path through a decision tree is controlled by questions like: is ‘ $x_1 > a?$ ’, ‘ $x_2 > b?$ ’, and so on. In contrast, NN models combine the predictors by a linear combination ($\sum_i w_i x_i$) before passing it through an activation function. With inhomogeneous predictors, for example temperature and pressure, treating the two separately as in decision trees makes more sense than trying to add the two together by a linear combination.

In environmental science, both structured and unstructured datasets are common. Examples of unstructured datasets include satellite images and 2-D or 3-D gridded output from weather/climate models. In the last few years, there has been rapid growth in the applications of DNN models to satellite images and weather/climate model output. Review/perspective papers covering DL in

² Prior to the advent of the convolutional layer, one usually had to rely on the linear technique of principal component analysis to compress the large number of input variables from images or 2-D grids, then supply a modest number of leading principal components as input to the NN to greatly reduce the number of model weights.

environmental science are found in C. Shen (2018), Reichstein et al. (2019), A. Y. Sun and Scanlon (2019) and in the edited book by Camps-Valls, Tuia, et al. (2021).

15.1 Transfer Learning

ML models, especially DNN models, generally require a large number of data points to train, which may not be available. Fortunately, in the *transfer learning* approach, ML models trained on a dataset with large sample size can transfer their learning to a different problem hampered by a relatively small sample size. For example, consider the task of developing facial recognition software for a rare species of monkeys with fewer than 100 images available. If one trains a DNN model with the small dataset, it would perform poorly. However, training the same model using a million or more human facial images would give a good model for human facial recognition. In the transfer learning approach, one trains this human model further with the small number of monkey images, and gets a much better model for monkey recognition. Basically, after the model has learned to recognize human faces well, its weights only need a moderate amount of tuning using the monkey images for it to adapt to the monkey problem. Transfer learning started as early as 1976 (Bozinovski, 2020), with a 1997 special issue of Machine Learning devoted to the subject (Pratt and Thrun, 1997).

Since DNN models usually require a large amount of data and computer time to train, many pre-trained DNN models developed by leading DL groups have been made available to the public. Such models have already been trained on large datasets and have learned to recognize patterns/objects. Researchers would then further train the pre-trained DNN with data particular to their areas of interest. Thus, transfer learning is able to greatly reduce the amount of training time and data for DNN models, thereby popularizing the use of DNN models. For instance, to detect *beluga whales* from passive acoustics monitoring, Zhong et al. (2020) used transfer learning to train CNN models to successfully classify input acoustic signals as belonging to the belugas or not. As the images used for transfer learning were from objects (e.g. dogs and cars) unrelated to acoustic signals, the model was able to learn how to extract features from these images, before it was further trained with relevant acoustic data.

In environmental science, the application of ML methods to climate problems has been impeded by relatively short observational records, as the long timescales in climate problems preclude a large number of independent data points in time. However, there are many climate models (a.k.a. general circulation models or dynamical models) that can simulate climate on a computer for centuries, millennia or longer. These large climate model datasets offer an exciting opportunity for transfer learning, that is, ML models can be first trained using the climate model data and then further trained using the short observational data records.

In interannual variability, prediction of the El Niño-Southern Oscillation (ENSO) by a CNN model was made by Ham et al. (2019). Transfer learning

was used to overcome the limitation of short observational records, which had hampered earlier NN models (Tang et al., 1997; B. Tang, Hsieh, et al., 2000; Wu, Hsieh, and B. Tang, 2006). For each target month of the year, the CMIP5 climate model data for 2,961 months (from the Coupled Model Intercomparison Project (CMIP), World Climate Research Programme) were first used to train the CNN, followed by additional training using 103 months of reanalysis data (between 1871 and 1973). The prediction model was tested using the reanalysis data from 1984 to 2017, where the 10-year gap between the final year in the training period and the earliest year in the test period was to prevent oceanic memory from the training period influencing the test period. During the test period, the all-season correlation skill of the Niño 3.4 index of the CNN model was much higher than those of current state-of-the-art dynamical forecast systems. In other words, by using a CNN to learn the ENSO behaviour from the dynamical models, then transfer the learning to observed data (i.e. reanalysis data), this transfer learning approach yielded a CNN with better accuracy in ENSO prediction than the dynamical models.

15.2 Convolutional Neural Network

Biological studies in visual cortexes pioneered by David H. Hubel and Torsten N. Wiesel in 1959 (which led to the 1981 Nobel Prize in Physiology or Medicine) inspired the development of the convolutional layer, first introduced as the neocognitron (Fukushima, 1975, 1980), later evolving into the convolutional neural network (CNN) in DL (LeCun, B. Boser, et al., 1989). In convolutional layers, a neuron is only connected to a small patch of neurons in the preceding layer, greatly reducing the number of weights compared to the traditional fully-connected layers in MLP NN models. For recognizing objects in images, Urban et al. (2017) empirically found that shallow NN and shallow CNN were unable to attain the accuracy of deep CNN models.

15.2.1 Convolution Operation

We have encountered the convolution operation previously – (11.25) in Section 11.2 on windows under time series analysis. In CNN, *discrete* convolution is performed by applying a *filter* (a.k.a. *kernel* or *mask*) $\mathbf{F}$ to an array $\mathbf{A}$, yielding an output array $\mathbf{B}$, with elements

$$B(i, j) = (A * F)(i, j) = \sum_k \sum_l A(k, l) F(i - k, j - l), \quad (15.3)$$

with the summation indices starting from $k = 0$ and $l = 0$. Equivalently, as the convolution operation is commutative ($\mathbf{A} * \mathbf{F} = \mathbf{F} * \mathbf{A}$),

$$B(i, j) = (F * A)(i, j) = \sum_k \sum_l F(k, l) A(i - k, j - l). \quad (15.4)$$

It is sometimes more convenient to express the convolution operation using a *correlation* operator,

$$B(i, j) = (F \star A)(i, j) = \sum_k \sum_l F'(k, l) A(i + k, j + l), \quad (15.5)$$

where $\mathbf{F}'$ is the *flipped filter* or flipped kernel, obtained by reversing the order of the rows and the columns of $\mathbf{F}$. For example, in Fig. 15.2, $\mathbf{F}$ and $\mathbf{F}'$ are given by

$$\mathbf{F} = \begin{pmatrix} 0 & 0 & 1 \\ -2 & 0 & 2 \\ 0 & -1 & 0 \end{pmatrix}, \quad \mathbf{F}' = \begin{pmatrix} 0 & -1 & 0 \\ 2 & 0 & -2 \\ 1 & 0 & 0 \end{pmatrix} \quad (15.6)$$

(note $\mathbf{F}'$ is not the same as the transpose of $\mathbf{F}$).

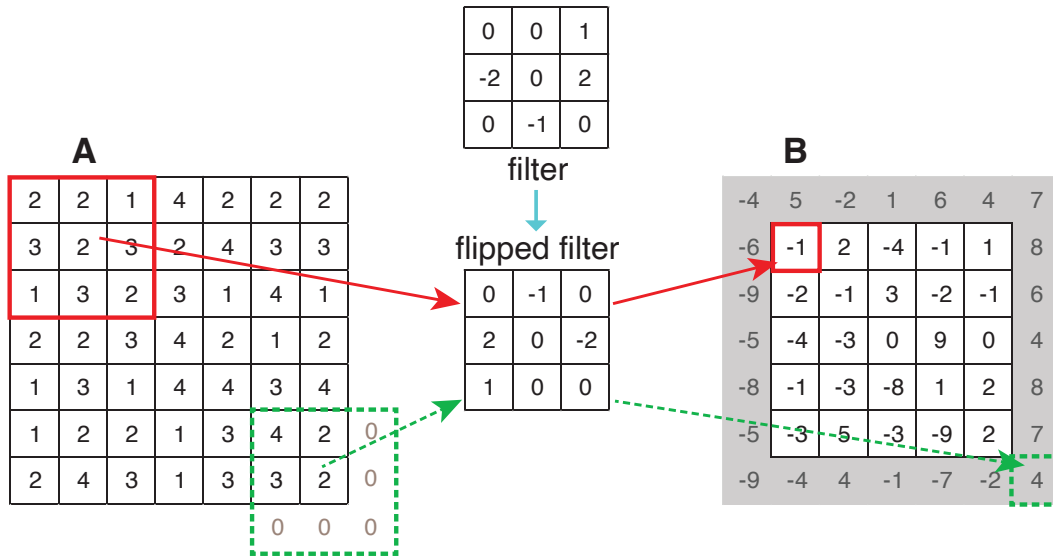


Figure 15.2 Convolution operation as illustrated by applying a 3×3 filter to a 7×7 matrix. The filter $\mathbf{F}$ is first flipped (both rows and columns) to give the flipped filter $\mathbf{F}'$. Nine elements of the input matrix $\mathbf{A}$ on the left are multiplied by the nine elements of the flipped filter, then summed and placed in the output matrix $\mathbf{B}$ on the right. If $\mathbf{B}$ is to remain the same size as $\mathbf{A}$, zeros must be padded outside the boundary of $\mathbf{A}$ to produce the extra elements shaded in grey (e.g. the dashed pixel).

In Fig. 15.2, $\mathbf{A}$ is a 7×7 matrix and $\mathbf{F}$ and $\mathbf{F}'$ are 3×3 matrices. Using (15.5), the convolution is simply the dot product between the nine points of $\mathbf{F}'$ and a patch of nine data points in $\mathbf{A}$. For the top left 3×3 patch in $\mathbf{A}$, the convolution gives

$$\begin{aligned} & 0 \times 2 - 1 \times 2 + 0 \times 1 + \\ & 2 \times 3 + 0 \times 2 - 2 \times 3 + \\ & 1 \times 1 + 0 \times 3 + 0 \times 2 = -2 + 6 - 6 + 1 = -1. \end{aligned} \quad (15.7)$$

To generate the rest of $\mathbf{B}$, the nine-point patch in $\mathbf{A}$ is shifted both horizontally and vertically, one grid or pixel at a time (i.e. using a *stride* of 1), and a dot product is taken with $\mathbf{F}'$. $\mathbf{B}$ has a smaller size than $\mathbf{A}$ – here the 7×7 matrix $\mathbf{A}$ is reduced to a 5×5 matrix $\mathbf{B}$. In general, for an $\mathbf{A}$ matrix of width m using a filter of width f , the output $\mathbf{B}$ has width $m - f + 1$. For example, If we use a 5×5 filter in Fig. 15.2, $\mathbf{B}$ would be reduced to a 3×3 matrix.

The stride s is a hyperparameter. If $s = 2$ is used, the patch over $\mathbf{A}$ to be operated on by $\mathbf{F}'$ is shifted by two pixels or grids, as in Fig. 15.3. The width of the matrix $\mathbf{B}$ becomes $(m-f)/s+1$, so increasing s can greatly reduce the output matrix size. In many DNN applications, the input consists of high-dimensional images, but the output is of much lower dimension, for example ‘storm’ or ‘no storm’, so it is useful to reduce the dimension as the signal propagates through the layers of a DNN.

At the boundary of $\mathbf{A}$, one has the option of padding with zeros beyond the boundary, so matrix $\mathbf{B}$ can remain the same size as $\mathbf{A}$ (Fig. 15.2). If z number of zeros are padded beyond a boundary, the width of the matrix $\mathbf{B}$ is

$$\frac{m - f + 2z}{s} + 1. \quad (15.8)$$

After $\mathbf{B}$ is generated by the convolution operation, there are two more steps: the same bias or offset is added to each element of $\mathbf{B}$, and the adjusted value is passed through an activation function, usually the rectified linear unit (ReLU), onto the next layer of neurons. The next layer can be another convolutional layer or a different type of layer. Details of the computations in CNN models are provided by Dumoulin and Visin (2018).

A 2-D image of length m_L and width m_W often uses three colour *channels* (RGB), so the input is a 3-D array ($m_L \times m_W \times 3$). This third dimension is called the *depth* of the layer – not to be confused with the *depth of the model*, which is the number of layers (with adjustable weights) in the CNN. The filter is then also 3-D, $f \times f \times 3$. In general, the input $\mathbf{A}$ can have m_D channels, that is, $\mathbf{A}$ is of dimension $m_L \times m_W \times m_D$. The output $\mathbf{B}$ can have m'_D channels. The convolution operation in (15.5) is generalized to

$$B(i, j, d') = \sum_d \sum_k \sum_l F'(k, l, d, d') A(i + k, j + l, d), \quad (15.9)$$

where d and d' are the indices for the channels in $\mathbf{A}$ and $\mathbf{B}$, respectively. The number of weights used to get $\mathbf{B}$ is

$$n_{\mathbf{B}} = f \times f \times m_D \times m'_D. \quad (15.10)$$

The weights in a filter are found by optimizing an objective function. Usually, the same filter is used for all the elements of $\mathbf{B}$, which allows a very economic sharing of weights compared to the traditional MLP NN. It is possible to not

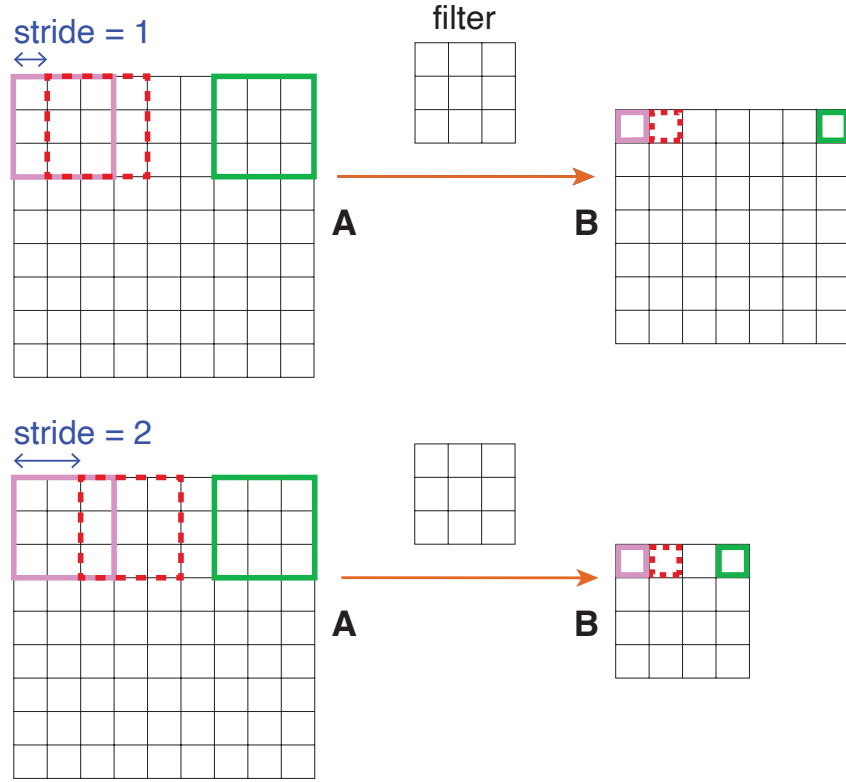


Figure 15.3 Effect of moving the (flipped) filter over the input matrix **A** at different stride s . **A** is of size 9×9 and the filter 3×3 , while the output **B** is of size 7×7 (for $s = 1$) and 4×4 ($s = 2$).

share the same filter for all elements of **B** (Goodfellow, Bengio, et al., 2016, section 9.5), which leads to more weights but also more flexibility. Even with the same filter weights used for all elements of **B**, the model is flexible as one can use more sets of filters, that is, increasing the channel depth m'_D .

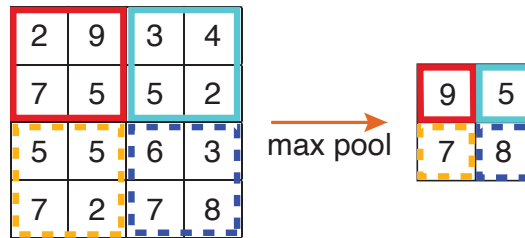
15.2.2 Pooling ☹️

Pooling is another operation used to *downsample*, that is, reduce the dimension of an array **A**, by extracting a single number from a patch of size $f \times f$ in **A**. The most common pooling used is maximum pooling or *max pooling*, where the filter simply extracts the maximum value from the patch. *Average pooling*, which returns the mean value of the patch, was used historically. While it has largely been replaced by max pooling, Weyn, Durran, and Caruana (2020) switched to average pooling from their earlier use of max pooling (Weyn, Durran, and Caruana, 2019).

Max pooling is most commonly performed over a 2×2 patch (Fig. 15.4), which reduces the width of **A** by half. When counting the number of layers in a CNN model, max pooling is not counted as a layer since it has no adjustable weights.

Max pooling can also be done with a stride $s \neq f$, with the width of the output $\mathbf{B}$ being $(m - f)/s + 1$. For example, if the input width is $m = 55$ pixels, $f = 3$ and $s = 2$, the output width is 27 (Krizhevsky et al., 2012, figure 2). With $s < f$, a pooling patch overlaps with its neighbouring patch, so this is called overlapping pooling, in contrast to the non-overlapping pooling in Fig. 15.4 with $s = f = 2$.

Figure 15.4 Example of a 4×4 input array undergoing a max pooling operation, where the output is the maximum value from each 2×2 patch. Here, the filter width $f = 2$ and the stride $s = 2$.



15.2.3 AlexNet ☹️

The first DNN to have a major impact was AlexNet, which won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2012 competition (Krizhevsky et al., 2012). The eight-layer AlexNet CNN model is composed of five convolutional layers followed by three *dense layers* (i.e. the traditional fully connected layers) (Fig. 15.5). Input images have 227×227 pixels (not 224×224 as given in Krizhevsky et al. (2012)), with three colour channels (RGB). Applying convolution using an 11×11 filter with a stride of $s = 4$ led to an array of 55×55 nodes in layer 1. Using 96 such filters led to 96 channels, with the number of weights in layer 1 being $11 \times 11 \times 3 \times 96 = 34,848$. There are an additional 96 weights from the bias or offset (one for each channel), so the total number of weights in layer 1 = 34,944. Increasing the number of channels in a CNN layer increases the number of weights at that layer, thereby increasing the model complexity. There are about 62 million weights/parameters in AlexNet, and the model used roughly 1.2 million training images, 50,000 validation images, and 150,000 testing images. Since the number of weights greatly exceeded the number of available images, data augmentation (Section 15.2.5) and dropout were used to avoid overfitting.

How does CNN actually detect an object from an image?

The learned features in the first layer of representation typically represent the presence or absence of edges at particular orientations and locations in the image. The second layer typically detects motifs by spotting particular arrangements of edges, regardless of small variations in the edge positions. The third layer may assemble motifs into larger combinations that correspond to parts of familiar objects, and subsequent layers would detect objects as combinations of these parts. The key aspect of deep learning is that these layers of features are not designed by human engineers: they are learned from data

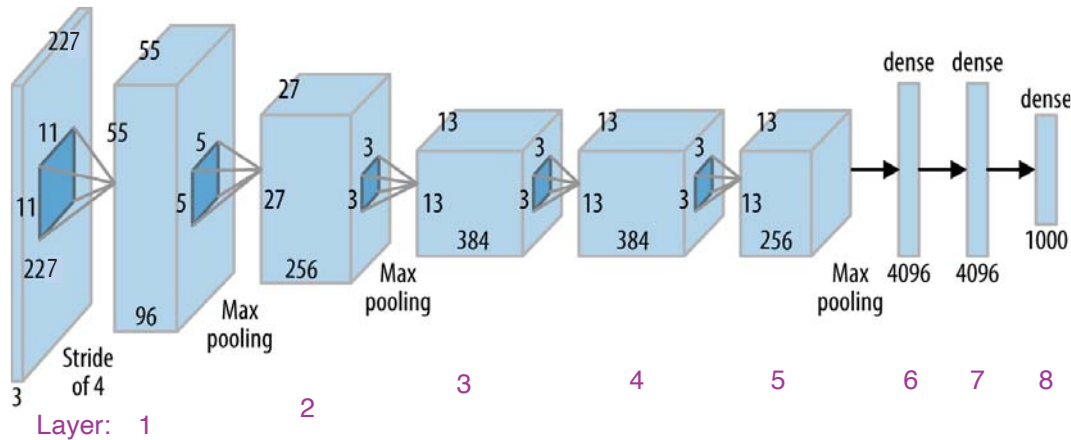


Figure 15.5 The eight-layer AlexNet CNN model has input images of 227×227 pixels and three colour channels (RGB), with the number of channels indicated by the depth (i.e. thickness) of the input block. Convoluting the input by a 11×11 filter with a stride of $s = 4$ led to an array of 55×55 , and the use of 96 such filters led to 96 channels in layer 1. After max pooling, convoluting by a 5×5 filter (with $z = 2$ for zero padding) led to the layer 2 array of size 27×27 , with 256 channels. All max pooling operations are done using a 3×3 filter with $s = 2$. Again max pooling, then convoluting by a 3×3 filter (with $z = 1$) led to the layer 3 array of 13×13 , with 384 channels. Convoluting with a 3×3 filter (with $z = 1$) led to the layer 4 array of 13×13 , with 384 channels, and convoluting again led to layer 5 of 13×13 , with 256 channels. After max pooling, the resulting 6×6 array with 256 channels is reshaped into a 1-D array of 4,096 nodes and is fully or densely connected to layer 6 with 4,096 nodes, which is fully connected to layer 7 with 4,096 nodes. Layer 7 is fully connected to layer 8 with 1,000 nodes, with the softmax activation function indicating which one of the 1,000 classes (e.g. cats, dogs, cars, etc.) the output belongs to. [Adapted from *TensorFlow for Deep Learning*, by Bharath Ramsundar and Reza Bosagh Zadeh. Copyright © 2018 Reza Zadeh, Bharath Ramsundar. Published by O'Reilly Media, Inc. Used with permission.]

using a general-purpose learning procedure. (LeCun, Bengio, et al., 2015)

The reason why CNN can use a filter with only a modest number of weights on an entire image is because the local statistics and signals in images are usually invariant to location. If a motif or pattern can appear in one part of the image, it could appear elsewhere, hence one can use the same filter weights for every node in the array, which allows CNN to greatly reduce the number of weights compared to the traditional fully connected NN.

Progress in CNN skills in image recognition continued: ZFNet, a modified AlexNet, won the ILSVRC 2013 competition (Zeiler and Fergus, 2014). For the

first layer, instead of a 11×11 filter and a stride of 4 in AlexNet, ZFNet used a 7×7 filter and a stride of 2. Using a CNN with 22 layers, GoogLeNet (from Google, with ‘LeNet’ imbedded in the name to pay tribute to the original LeNet CNN from LeCun, B. Boser, et al. (1989)) won the ILSVRC 2014 competition (Szegedy et al., 2015). The first runner-up at this competition was VGGNet, from the Visual Geometry Group (VGG) at the University of Oxford (Simonyan and Zisserman, 2015). VGGNet has variants using 16 or 19 layers.

15.2.4 Residual Neural Network (ResNet)

For the winners in the ILSVRC competitions, as the number of layers in the CNN increased from 8 in 2012 to 22 in 2014, eventually the problem of vanishing gradients again put an upper limit on the number of layers. The *residual neural network* (ResNet) from Microsoft introduced *skip connections* (i.e. shortcuts to jump over some layers) to overcome this problem. Using models with up to 152 layers, ResNet came first in the ILSVRC 2015 competition (He et al., 2016). The ResNet architecture even allowed a model with 1,202 layers to perform fairly well.

Figure 15.6 illustrates a skip connection, where the output from layer l is directly connected to the input of layer $l + 3$. Layer $l + 3$ thus receives the output from layer $l + 2$ and from layer l . Bigger steps are also allowed in skip connections, mapping layer l directly to layer $l + k$ ($k \geq 3$). The use of skip connections is further extended in *densely connected convolutional networks* (DenseNet) (G. Huang, Z. Liu, et al., 2017).

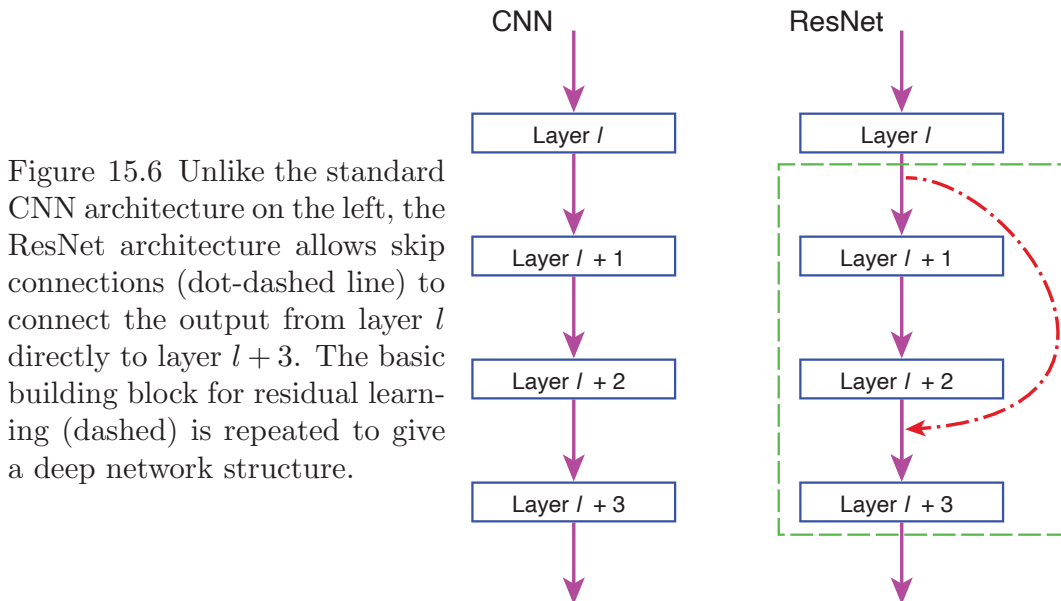


Figure 15.6 Unlike the standard CNN architecture on the left, the ResNet architecture allows skip connections (dot-dashed line) to connect the output from layer l directly to layer $l + 3$. The basic building block for residual learning (dashed) is repeated to give a deep network structure.

The term ‘residual’ comes from using the NN layers to model the residual $\mathbf{F}(\mathbf{x}) = \mathbf{H}(\mathbf{x}) - \mathbf{x}$, where $\mathbf{H}(\mathbf{x})$ is the desired underlying mapping. In other words, one tries to remove the linear signal from $\mathbf{H}(\mathbf{x})$, so the NN layers can

focus on modelling the non-linear residual signal, as it is inefficient for non-linear NN layers to model a linear signal. The skip connection allows the linear signal to bypass³ the non-linear NN layers in the residual learning block (Fig. 15.6).

15.2.5 Data Augmentation ☐😊

In many applications, the number of available training images is quite limited, too few to properly train the large number of weights in deep CNN models. We have seen the use of transfer learning in Section 15.1 to alleviate this problem. Furthermore, one can generate more training images by various *data augmentation* techniques. For instance, translation, rotation, flipping, geometric scaling, colour modification, cropping, noise injection and random erasing can be used to generate more training images (Shorten and Khoshgoftaar, 2019). One should use only transformations that give physically realistic images. For instance, shifting a tropical cyclone to polar latitudes or flipping the cyclone (i.e. reversing the anticlockwise circulation to clockwise circulation in the Northern Hemisphere) would supply irrelevant training data for the CNN model.

A more advanced approach to data augmentation is to use generative adversarial networks (GAN) (Section 15.5) to produce highly realistic fake images. The GAN model has two adversarial components: the generative network creates fake images, while the adversarial network tries to determine if the image is fake or genuine. The competition between the adversarial components leads to increasingly more realistic fake images.

15.2.6 Applications in Environment Science ☐😊

In the last few years, there have been many papers applying CNN models to environmental science. To detect extreme weather patterns from 2-D images of atmospheric variables, CNN models composed of two convolutional layers followed by two fully connected layers were used by Y. Liu, Racah, et al. (2016), with three separate models built to estimate the posterior probability of three types of extreme events – tropical cyclones, atmospheric rivers and weather fronts. A deep CNN model with six convolutional layers followed by two fully connected layers was used by Lagerquist, Ryan and McGovern, Amy and Gagne, II, D. J. (2019) to detect synoptic-scale weather fronts, where the three output nodes with softmax activation functions gave the probabilities for cold front, warm front and no front. For next-hour tornado prediction, with radar images as input, Lagerquist et al. (2020) used a deep CNN model to output the probability of a tornado.

For automated detection of weather fronts, Baird and Kunkel (2019) used a four-layer CNN model, where three convolutional layers followed by one fully connected layer using the softmax action function (with all layers having the same spatial dimension) gave the posterior probability for five frontal categories (cold, warm, stationary, occluded and none) at each pixel (unlike the previous

³ Linear bypass NN architecture has also been used in shallow NN models (Section 6.4.2).

two studies where the output had no spatial resolution). Assigning a class label to each pixel of an image is called *semantic segmentation*. Most CNN models for semantic segmentation use an encoder–decoder structure (Section 15.3).

To predict the El Niño–Southern Oscillation (ENSO) phenomenon, Ham et al. (2019) used CNN to predict the Niño 3.4 sea surface temperature (SST) anomaly index. The input were the 2-D anomaly maps of SST and upper ocean heat content at time t , $t - 1$ and $t - 2$ months (i.e. six input channels of 24×72 pixel images). The four layers were three convolutional layers followed by a fully connected layer, with the output 1-D vector containing Niño 3.4 at time t , $t + 1$, $\dots$, $t + 22$ months. The model was pre-trained using long records from CMIP5 climate model output as described in Section 15.1 on transfer learning.

Training deep CNN with past weather data to predict future weather has been explored (Weyn, Durran, and Caruana, 2019; Weyn, Durran, and Caruana, 2020). For short- to medium-range forecasting, CNN has been able to significantly outperform persistence, climatology and a coarse-resolution dynamical numerical weather prediction (NWP) model, though underperforming a high-resolution state-of-the-art operational NWP system. The model has also been extended to sub-seasonal forecasting (Weyn, Durran, Caruana, and Cresswell-Clay, 2021).

For researching the occurrence and movement of the *humpback whale*, A. N. Allen et al. (2021) used a ResNet CNN to automatically detect humpback whale song in a passive acoustic dataset with 13 monitoring sites in the North Pacific over a 14-year period.

15.3 Encoder–Decoder Network

The encoder–decoder NN architecture has served as the foundation in autoencoders (and its applications to non-linear principal component analysis) (Sections 10.5 and 10.6) under unsupervised learning. The arrival of the CNN architecture has greatly advanced encoder–decoder networks and autoencoders. The encoder–decoder structure is also used in *supervised* learning, for example in semantic segmentation, where a class label is assigned to each pixel of an output image, or in regression problems with real values assigned to each output pixel. The encoder structure funnels the high-resolution input image(s) to a bottleneck, thereby extracting the important information but ignoring the unimportant details in the input. The decoder structure then expands from the low-resolution bottleneck back to higher-resolution image(s). Proceeding from lower resolution to higher resolution is called *upsampling*, opposite to downsampling operations such as max pooling.

Nearest neighbour upsampling is the simplest, where an input pixel value is reproduced over multiple pixels in the higher-resolution output (Fig. 15.7). To avoid the blocky appearance, bilinear interpolation upsampling is an alternative.